

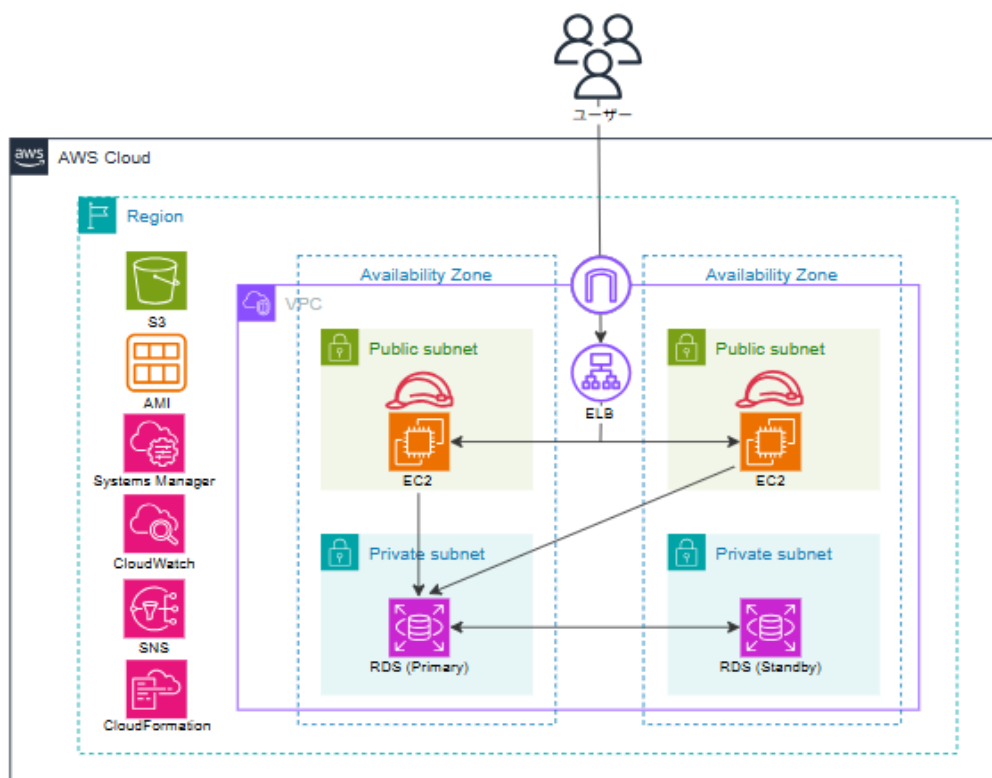
高可用なWebサイト構築

概要

初級編では、以下の内容を実施します。

- VPC、EC2、RDS、S3、ELBを利用して、
可用性の高いブログサイト（WordPress）を構築する
- EC2 Instance Connect/Systems Managerを使用したEC2インスタンスへのコマンド実行を行い、CloudWatch/CloudWatch Logsを利用したモニタリングを行う
- CloudFormation、Terraformを利用して、シンプルな構成のコードによる構築を行う

最終的には次の構成図で示すようなアプリケーション環境を構築します。



演習の流れ

環境の構築は次のステップに分けて取り組んでください。

- タスク1. VPCリソースの作成
- タスク2. EC2とRDSの作成
- タスク3. S3の作成とIAMロールの利用
- タスク4. ELBの作成と環境の冗長化
- タスク5. Systems ManagerとCloudWatchの利用
- タスク6. IaCによるリソースの作成

演習の進め方

次ページ以降に掲載されている「タスクの詳細」を元に、ドキュメントやインターネット上の情報を必要に応じて参照しながら構築を行ってください。

AWS環境を複数人で共有している場合は、必要に応じて重複しない一意の名称に置き換えて作業を進めてください。

調査を行ったうえで構築をすることが難しかった場合は、別途配布している各タスクの「手順書」を元に構築を行っていただくことが可能です。

各タスクの完了後は、別途配布している「AWS構築演習チェックシート」に各タスクの実施結果のセルフチェックと振り返りを記入し、成果物として講師へ提出してください。セルフチェックでは、以下の選択肢から各タスクを自己評価します。

- ✓ タスクおよびドキュメント等を元に構築を完了することができた
- ✓ 手順書を元に構築を完了することができた
- ✓ 構築を完了することができなかった

また、AWSを利用する上で無駄な料金が発生しないよう、**演習を終える際には必ずサービスを停止**してください。

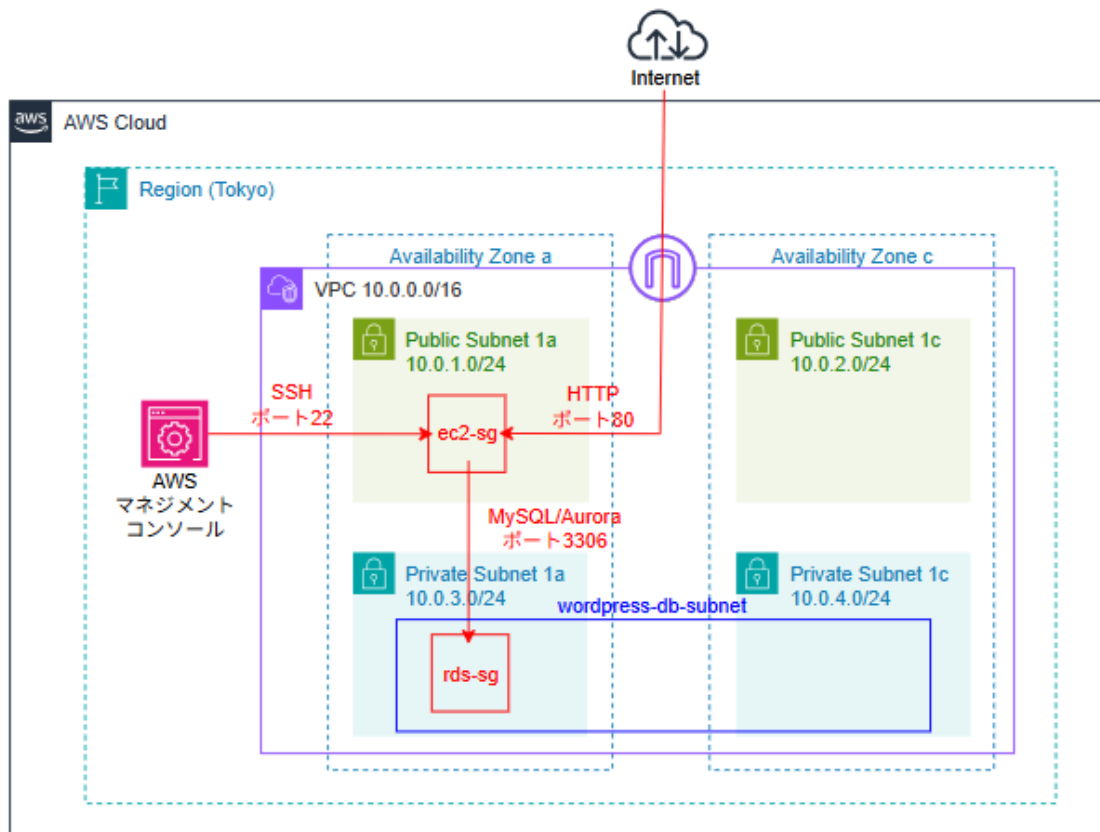
更に、演習で自身が使用しているリソース以外(案件で使用している検証環境など)には**決して触れない**ようお願いします。

タスクの詳細

タスク1. VPCリソースの作成

タスクのGOAL

- ✓ 必要なVPCリソースを作成する
- ✓ サービスを起動し、確認を行った後は、必ず都度サービスを停止させる
- ✓ タスク完了後のアーキテクチャーは以下の通りです



タスクの流れ

- 1-1. VPCとサブネットを作成する
- 1-2. EC2用のセキュリティグループを作成する
- 1-3. RDS用のセキュリティグループを作成する
- 1-4. RDS用のサブネットグループを作成する

タスクの要件

- 1-1. VPCとサブネットを作成する
 - リージョンは東京 (ap-northeast-1) を選択する
 - VPCの名称は「wordpress-vpc」とする
 - VPCのCIDRブロックは10.0.0.0/16とする

- 「ap-northeast-1a」と「ap-northeast-1c」の2つのアベイラビリティゾーンを利用する
- パブリックサブネットを2つ作成する
 - CIDRブロックは「10.0.1.0/24」「10.0.2.0/24」とする
 - サブネットに関連付けられたルートテーブルからインターネットゲートウェイへの経路があること
- プライベートサブネットを2つ作成する
 - CIDRブロックは「10.0.3.0/24」「10.0.4.0/24」とする
 - サブネットに関連付けられたルートテーブルからインターネットゲートウェイへの経路がないこと

1-2. EC2用のセキュリティグループを作成する

- 「ec2-sg」と命名する
- インバウンドルールでインターネットから80番ポートへのHTTPアクセスを許可する
- インバウンドルールでEC2 Instance Connectを利用した22番ポートへのSSHアクセスを許可する

※ここでは学習用に一部のセキュリティ設定を簡略化しています。実運用では、SSHの接続元を限定するなど、必要最小限のアクセス制御を行います。なお、後続のタスクで、実運用に近い構成も確認します。

1-3. RDS用のセキュリティグループを作成する

- 「rds-sg」と命名する
- インバウンドルールで ec2-sg から3306番ポートへのMySQL/Auroraアクセスを許可する

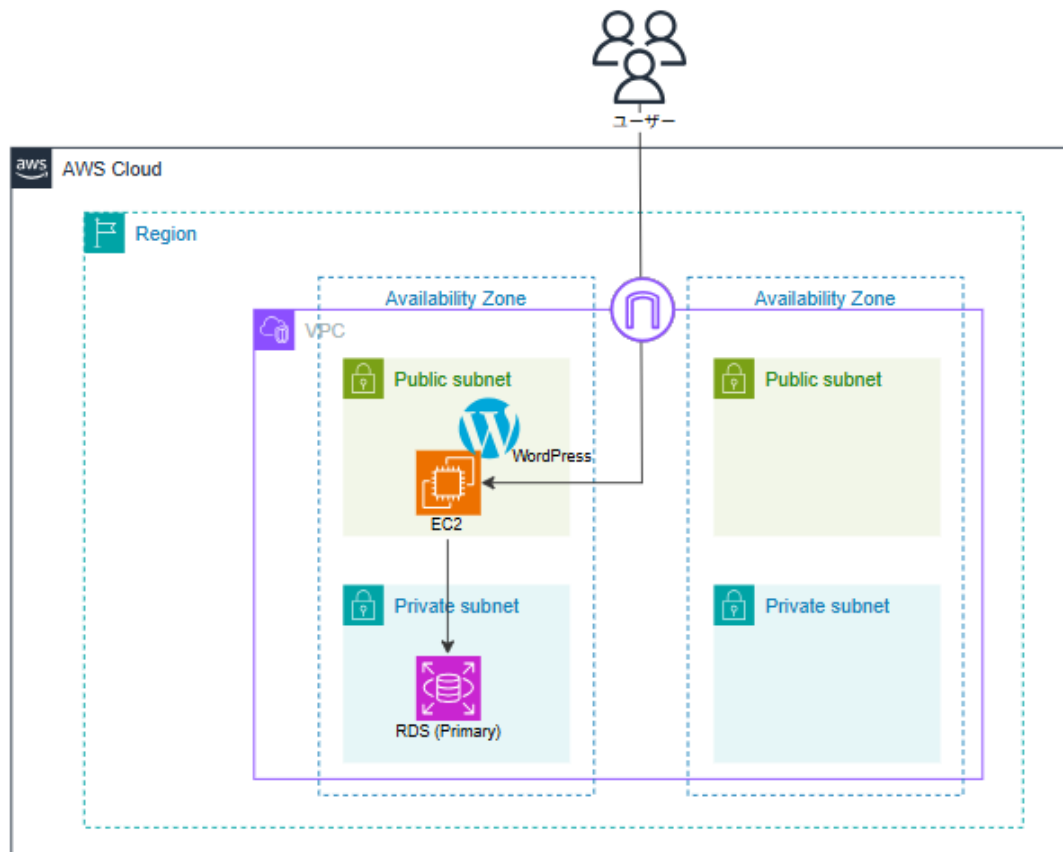
1-4. RDS用のサブネットグループを作成する

- サブネットグループを「wordpress-vpc」に配置する
- 「wordpress-db-subnet」と命名して2つのプライベートサブネットを指定する

タスク2. EC2とRDSの作成

タスクのGOAL

- ✓ RDSインスタンスとEC2インスタンスを作成してWordPressを構築する
- ✓ サービスを起動し、確認を行った後は、必ず都度サービスを停止させる
- ✓ タスク完了後のアーキテクチャは以下の通りです



タスクの流れ

- 2-1. RDSインスタンスを作成する
- 2-2. EC2インスタンスを作成する
- 2-3. WordPressを構築する

タスクの要件

2-1. RDSインスタンスを作成する

- DBエンジンは「MySQL」の「8.0.41」を利用する
- インスタンスクラスは「db.t3.micro」を利用する
- テンプレートは「無料利用枠」を利用する
- デプロイオプションは「シングルAZ DBインスタンスデプロイ（1インスタンス）」を利用する
- DBインスタンス識別子は「wordpress-rds」とする
- マスターユーザー名は「admin」とし、マスターパスワードはユーザーが指定したものもしくは自動生成したものを独自に管理する
- VPCは「wordpress-vpc」を利用する
- DBサブネットは「wordpress-db-subnet」を利用する
- セキュリティグループは「rds-sg」を利用する
- データベース認証は「パスワード認証」を利用する
- 最初のデータベース名は「wordpress」とする
- 自動バックアップはオフとする
- 以下の情報はデータベースへ接続する際に必要となるため控えておくこと
 - マスターユーザー名 と マスターパスワード
 - 最初のデータベース名

2-2. EC2インスタンスを作成する

- 名前は「WordPress 01」とする
- AMIは「Amazon Linux 2023 (kernel-6.12)」を利用する
- インスタンスタイプは「t3.micro」を利用する
- VPCは「wordpress-vpc」を利用する
- アベイラビリティゾーンは「ap-northeast-1a」を利用する
- セキュリティグループは「ec2-sg」を利用する

2-3. WordPressを構築する

- EC2 Instance Connectを利用してEC2インスタンスへ接続する
- 以下のコマンドを実行してWordPressを構築する

```
# システムパッケージをアップデート
sudo dnf update -y

# WordPress 実行に必要なになるPHPとライブラリをインストール
sudo dnf install php php-cli php-common php-mbstring php-xml php-gd php-mysqlnd php-curl php-zip -y

# MySQLレポジトリとGPGキー、クライアントのインストール
sudo dnf install https://dev.mysql.com/get/mysql80-community-release-el9-5.noarch.rpm -y
sudo rpm --import https://repo.mysql.com/RPM-GPG-KEY-mysql-2022
sudo dnf install mysql-community-client -y

# 初期インストールされているApacheの起動・自動起動設定
sudo systemctl enable httpd.service
sudo systemctl start httpd.service

# WordPressをダウンロードして解凍する
wget https://ja.wordpress.org/latest-ja.tar.gz
tar -xzf latest-ja.tar.gz

# Webサーバー上で公開するためのファイルの複製と権限設定
sudo cp -r ~/wordpress/* /var/www/html/
sudo chown apache:apache -R /var/www/html
```

- EC2インスタンスのパブリックIPアドレスへHTTPでアクセスし、データベース接続情報を入力して（テーブル接頭辞は「wp_」を指定する）データベースへ接続する
- 以下の情報を指定してWordPressをインストールする
 - サイトのタイトル：任意に指定する
 - ユーザー名：admin
 - パスワード：初期生成されているものを利用
(後で利用するため控えておくこと)
 - メールアドレス：自身のメールアドレス
 - 検索エンジンでの表示：チェックを入れる（検索されないようにする）

※ここでは、初期構築や動作確認を行いやすくするため、EC2 Instance Connect を利用して接続します。後続のタスクでは、Systems Manager を利用した接続やコマンド実行も確認します。

動作確認

最後に以下の手順に従ってWordPressを利用してサンプルのブログ記事を投稿してみましょう。

- WordPressのインストールが完了したら「ログイン」をクリックし、ユーザー名とパスワードでログインを行う
- WordPressの管理画面の左にあるメニューから [設定] > [パーマリンク] とクリックする
- パーマリンクの設定画面が表示されたら共通設定の一覧から [基本] を選択して [変更を保存] をクリックする
- メニューの [投稿] から [投稿を追加] をクリックする
- 投稿の編集画面が表示されたら任意のタイトルを入力する

- 続けて任意の本文を入力して改行を行い [+] ボタンを押して任意の画像を挿入する
- 画面右上にある [公開] を2回続けてクリックする
- [投稿を表示] を押して作成した記事を確認する
 - 記事内の画像を右クリックして別のタブで開くと画像のURLを確認できる
 - EC2 インスタンスで以下のコマンドを実行すると画像ファイルを確認することができます

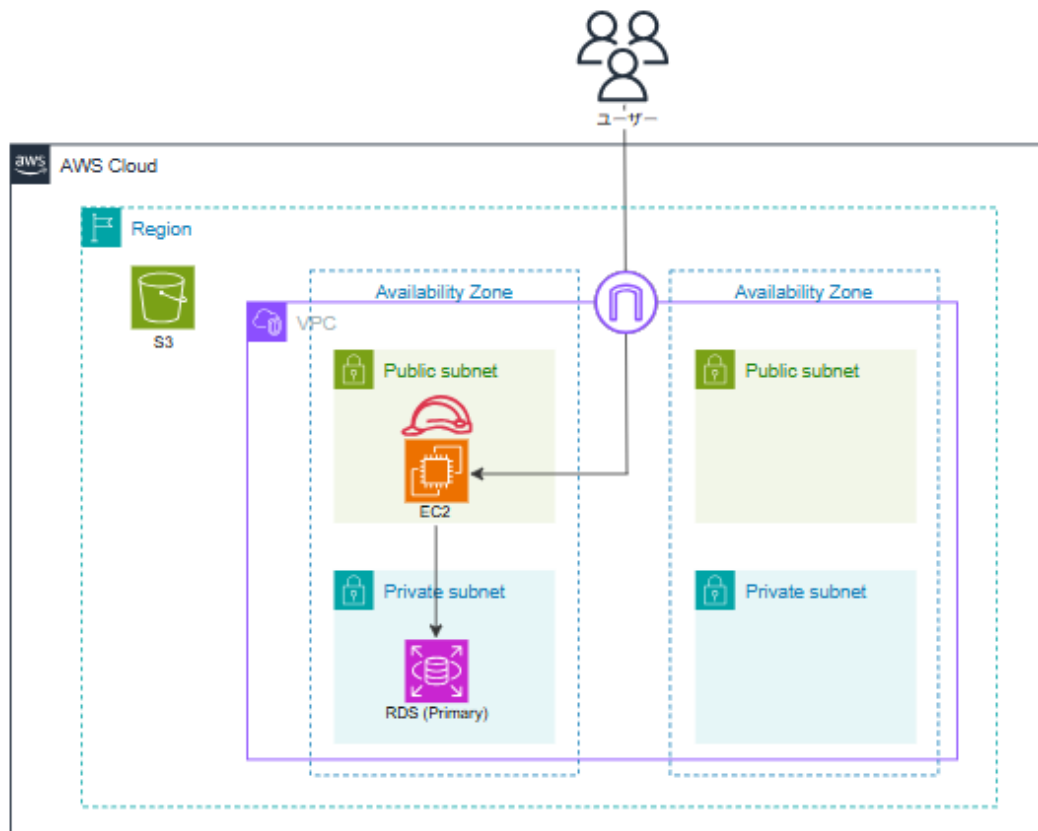
```
ls -la /var/www/html/wp-content/uploads/yyyy/mm/
```

※yyyy、mmは作業日に合わせて年と月を指定してください

タスク3. S3の作成とIAMロールの利用

タスクのGOAL

- ✓ S3バケットを作成してEC2インスタンスから利用可能にする
- ✓ サービスを起動し、確認を行った後は、必ず都度サービスを停止させる
- ✓ タスク完了後のアーキテクチャーは以下の通りです



タスクの流れ

- 3-1. S3バケットを作成する
- 3-2. IAMロールを作成しEC2インスタンスへ割り当てる
- 3-3. S3を利用するためのプラグインをWordPressへ追加する

タスクの要件

3-1. S3バケットを作成する

- バケット名はランダム文字列を利用して「wordpress-123456789」などと命名する
- 東京リージョンを利用する
- ACL有効を選択する
- 「パブリックアクセスをすべてブロック」のチェックを外す

3-2. IAMロールを作成しEC2インスタンスへ割り当てる

- ロール名は「ec2-wordpress-role」とする
- IAMポリシーは「AmazonS3FullAccess」を付与する
- 作成したIAMロールを「WordPress 01」インスタンスに割り当てる

3-3. S3を利用するためのプラグインをWordPressへ追加する

- 以下のURLを参考にWebブラウザからWordPress 01インスタンスのWordPress管理者ページへアクセスする
 - <http://ec2のパブリックIPアドレス/wp-login.php>
- ユーザー名とパスワードを指定してログインする
 - ユーザー名：admin
 - パスワード：WordPressの作成時に控えたパスワード
- 管理画面の左側にあるメニューから [プラグイン] > [プラグインを追加] とクリックする
- 検索窓に [WP Offload Media Lite for Amazon S3] と入力して検索を行い、表示された [Amazon S3、DigitalOcean Spaces、Google Cloud Storage用のWP Offload Media Lite] の欄にある [今すぐインストール] をクリックする
- インストールが完了するとインストールボタンが「有効化」に変わるので [有効化] をクリックする

- 遷移したプラグインの一覧画面で、WP Offload Media Liteの[Settings]をクリックする
- 「1.Select Provider」欄で「Amazon S3」のラジオボタンを選択する
- 「2.Connection Method」欄で「My server is on Amazon Web Services and I'd like to use IAM Roles」のラジオボタンを選択して、[Save&Continue] をクリックする
- 「1.New or Existing Bucket?」欄で「Use Existing Bucket」を選択する
- 「2.Select Bucket」欄で[Browse existing buckets] を選択する
- 利用可能な S3 バケットの一覧が表示されたら自身で作成した S3 バケットを選択して [Save Selected Bucket] をクリックする
- 遷移後の画面はデフォルトのまま「Keep Bucket Security As Is」をクリックする

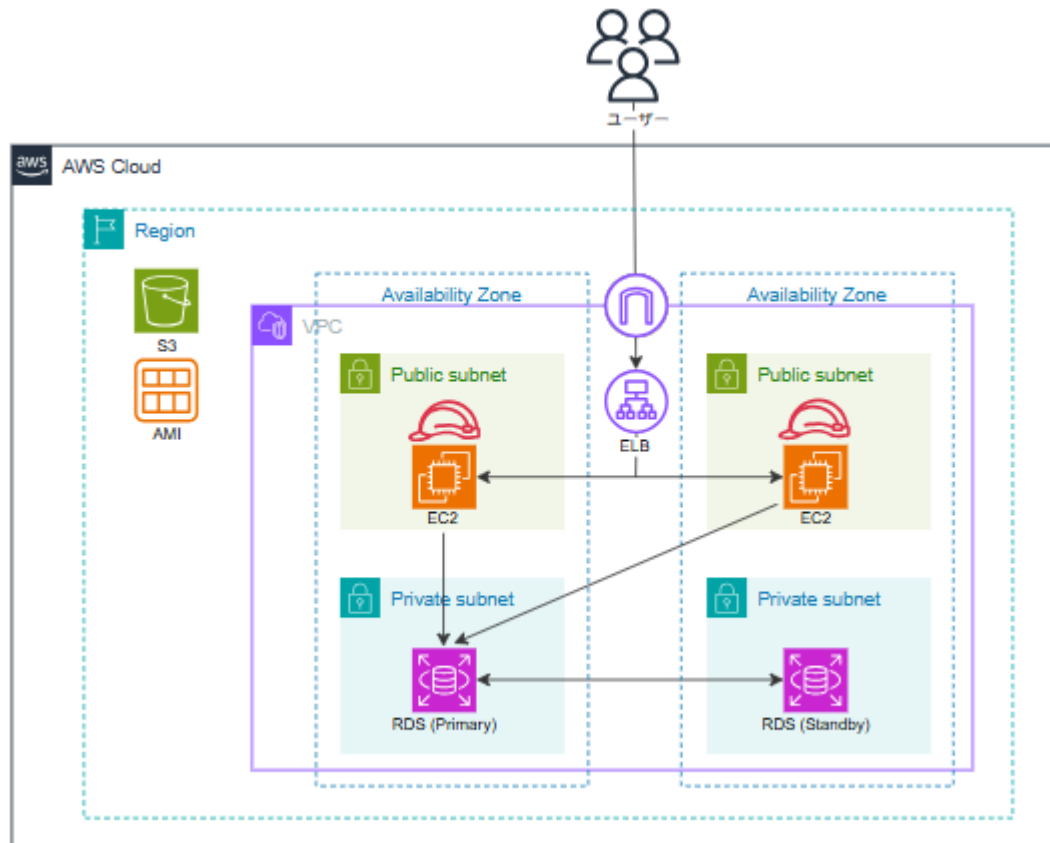
動作確認

- WordPress の管理画面で、画面左のメニューから [投稿] > [手順追加] をクリックする
- 記事のタイトルと本文を入力し、[+] ボタンをクリックして画像を挿入する
※このとき挿入する画像は新規で新しくアップロードしてください
- 記事の作成が終わったら、画面右上にある [公開] を2回続けてクリックする
- [投稿を表示] をクリックして投稿した記事を表示する
- 記事内の画像を右クリックして別タブで開くと、画像の URL が S3 バケットの URL であることを確認できます

タスク4. ELBの作成と環境の冗長化

タスクのGOAL

- ✓ ELBやAMIを利用してマルチAZ環境を構築する
- ✓ サービスを起動し、確認を行った後は、必ず都度サービスを停止させる
- ✓ タスク完了後のアーキテクチャは以下の通りです



タスクの流れ

- 4-1. ALBを作成する
- 4-2. カスタムAMIを作成してEC2を複製する
- 4-3. RDSをマルチAZ化する

タスクの要件

4-1. ALBを作成する

- 「wordpress-alb」という名前でApplication Load Balancerを作成する
- VPCは「wordpress-vpc」を利用する
- サブネットはすべてのパブリックサブネットを選択する
- セキュリティグループは新しく以下を作成する
 - 「elb-sg」と命名する
 - インバウンドルールでインターネットからポート80番へのHTTPアクセスを許可する
- リスナーは「wordpress-tg」という名前でターゲットグループを作成し、ターゲットに「WordPress 01」インスタンスを追加する
- ヘルスチェックには以下の値を指定する
 - プロトコル：HTTP
 - ヘルスチェックパス：/wp-login.php
 - 正常のしきい値：2
 - 間隔：30

※ここでは動作確認を優先した構成としています。実運用では、EC2 への通信はALB 経由に限定する構成を検討してください。なお、後続のタスクで、実運用に近い構成も確認します。

動作確認

- ALBの構築が完了したらロードバランサーのDNS名にHTTPアクセスを行い、WordPressの画面が表示されることを確認する
- 最後にELB経由でWordPressへアクセスした場合でも、正常にリクエストを処理できるように以下の手順でサイトのアドレス設定を変更します
 - WordPressの管理者用ページを表示して画面左のメニューから「設定」をクリックする

- 「WordPressアドレス(URL)」と「サイトアドレス(URL)」をELBのDNS名へ変更する

`http://<ロードバランサーのDNS名>`

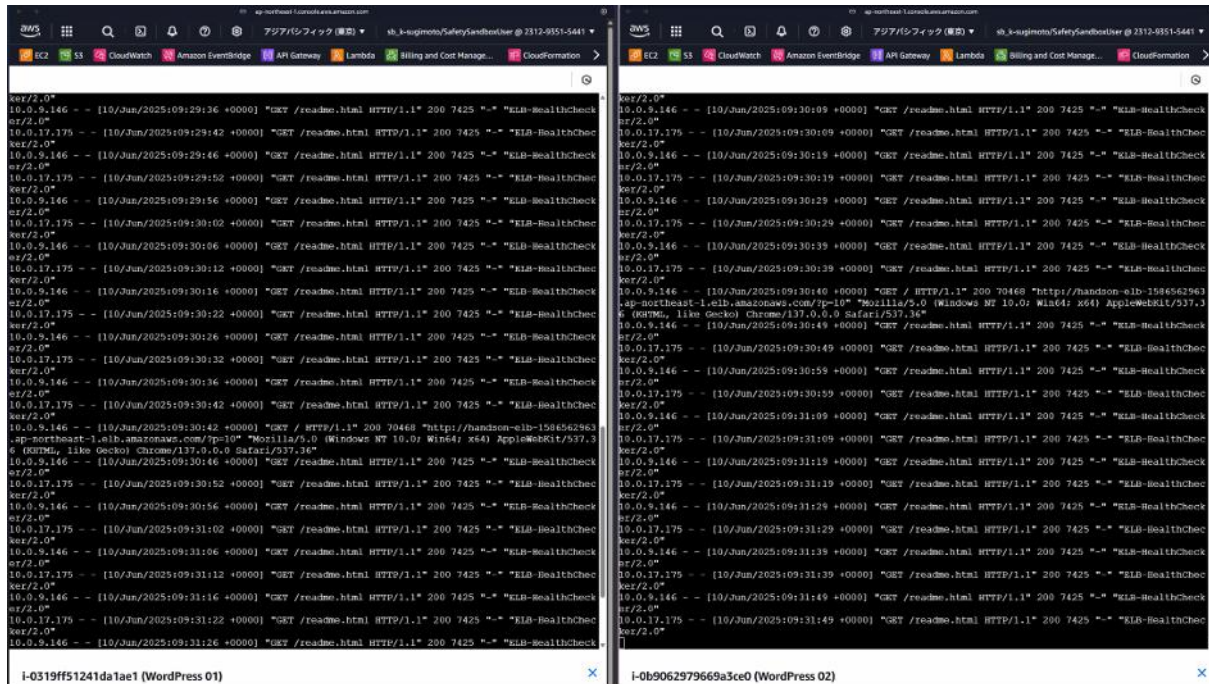
4-2. カスタムAMIを作成してEC2を複製する

- 「WordPress 01」 インスタンスを元にAMIを作成する
- 作成したAMIを元にEC2インスタンスを複製する
 - 名前は「WordPress 02」とする
 - インスタンスタイプはt3.microを利用する
 - VPCはwordpress-vpcを利用する
 - アベイラビリティゾーンはap-northeast-1cを利用する
 - セキュリティグループはec2-sgを利用する
 - パブリックIPアドレスの割り当てを有効にする
 - 必要に応じて、WordPress 01と同様のIAMロールを付与する
 - ロール付与後、必要なコマンドを実行して設定を反映する
- EC2の複製を行ったら「wordpress-tg」へターゲット登録を行います

動作確認

- ELBによってリクエストの振り分けが行われていることを確認するために、2つのタブを開いてSystems Managerのセッションマネージャーを利用して2つのEC2インスタンスへそれぞれアクセスします。
- アクセスできたら以下のコマンドを実行しましょう

```
sudo tail -f /var/log/httpd/access_log
```
- その状態でELBのDNS名に複数回アクセスすると、アクセスログがそれぞれのEC2インスタンスで出力されていることを確認できます



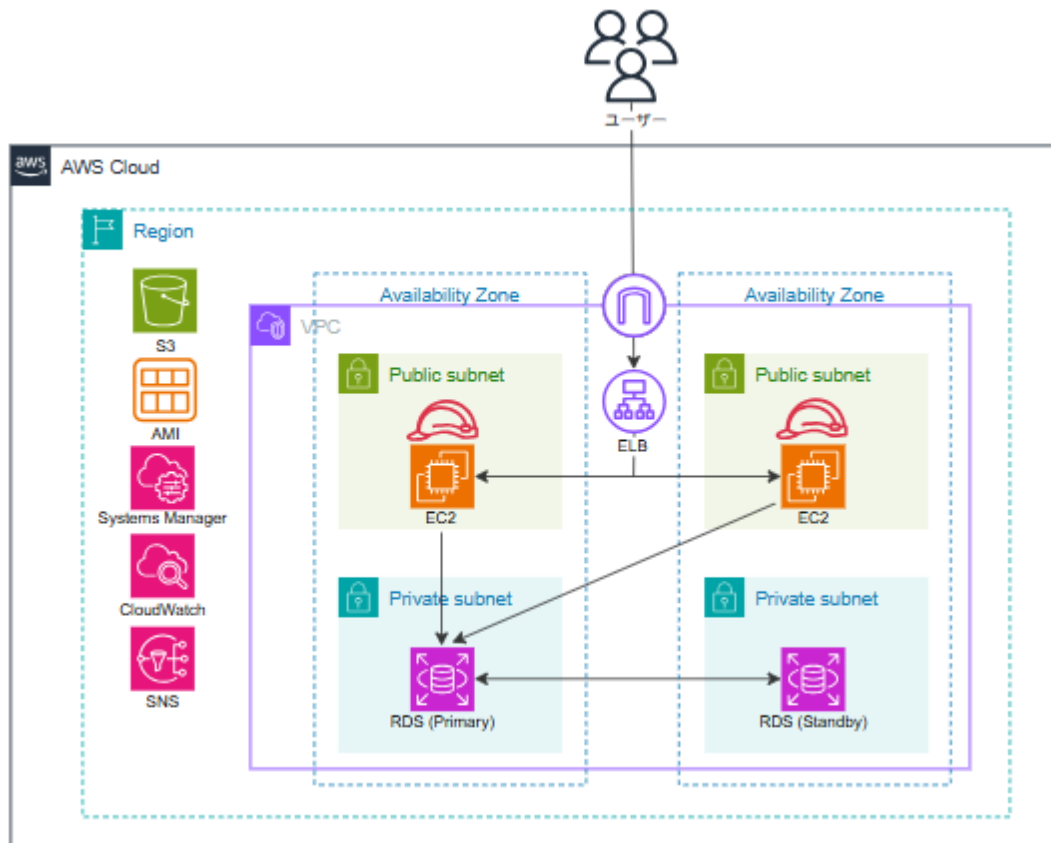
4-3. RDSをマルチAZ化する

- wordpress-rdsを元にマルチAZを利用してスタンバイインスタンスを作成する

タスク5. Systems ManagerとCloudWatchの利用

タスクのGOAL

- ✓ CloudWatchの利用設定を行ってカスタムメトリクスやログを監視する
- ✓ サービスを起動し、確認を行った後は、必ず都度サービスを停止させる
- ✓ タスク完了後のアーキテクチャは以下の通りです



タスクの流れ

- 5-1. IAMロールへIAMポリシーを追加する
- 5-2. Systems Managerを利用したCloudWatchエージェントの設定と起動
- 5-3. CloudWatchのメトリクスを利用したアラートを設定する
- 5-4. CloudWatch Logsを確認する

タスクの要件

5-1. IAMロールへIAMポリシーを追加する

- ec2-wordpress-roleへ以下を実行することができるIAMポリシーを追加します
 - Amazon CloudWatchへアクセスを行い CloudWatch Agentの設定をAWS Systems ManagerのParameter Storeに保存することができるAWSマネージドポリシー
 - Systems Managerを利用してEC2インスタンスへRun Commandを実行することができるAWSマネージドポリシー

5-2. Systems Managerを利用したCloudWatchエージェントの設定と起動

- Systems ManagerのRun Commandから「AWS-ConfigureAWSPackage」ドキュメントを2つのEC2インスタンスへ実行する
- Systems ManagerのParameter Storeにパラメーターを作成する
 - 名前は「AmazonCloudWatch-Agentconf」とする
 - 値は「AmazonCloudWatch-Agentconf.json」の内容を転記する
- Systems ManagerのRun Commandを利用してcollectdの利用準備を行うための以下のコマンドを2つのEC2インスタンスへ実行する

```
sudo mkdir -p /usr/share/collectd/; sudo touch /usr/share/collectd/types.db
```

- Systems ManagerのRun Commandから「AmazonCloudWatch-ManageAgent」ドキュメントを利用して2つのEC2インスタンス内のCloudWatch Agentを起動する

※ここからは、Systems Manager を利用して複数インスタンスへの接続やコマンド実行など、複数台を管理しやすい方法を確認します。

動作確認

- 数分待ってからCloudWatchダッシュボードにアクセスして画面左側のメニューの「すべてのメトリクス」から「CWAgent」を選択し、設定ファイルで指定したカスタムメトリクス項目が取得されていることを確認する

5-3. CloudWatchのメトリクスを利用したアラートを設定する

- WordPress 01 の CPU 使用率 メトリクスに対して、CPU使用率が 50% を超えた場合に自分のメールアドレスへアラームを発報する設定を行う
- 設定が完了したら WordPress 01へ接続して以下のコマンドを実行してアラートが発砲されることを確認する

```
#簡易負荷ツールをインストール
```

```
sudo amazon-linux-extras install -y epel; sudo yum install -y stress
```

```
#600 秒間 CPU 負荷をかける
```

```
stress -c 1 -t 600
```

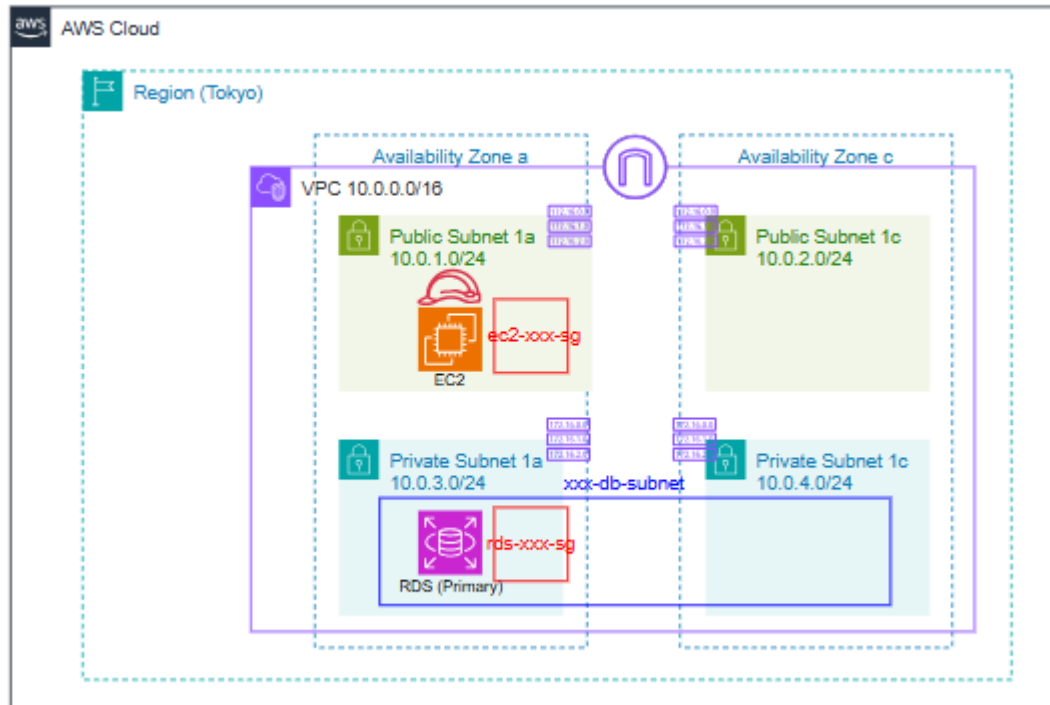
5-4. CloudWatch Logsを確認する

- ロードバランサーのDNS名へ複数回アクセスを行い、CloudWatchのロググループにログストリームが作成されていることを確認する
- CloudWatchのLogs InsightsからHTTPAccessLogを利用してブログサイト内でリクエスト数が多い URL Top10 を表示するクエリを実行する

タスク6. IaCによるリソースの作成

タスクのGOAL

- ✓ CloudFormationテンプレートおよびTerraformを利用してリソースを作成する
- ✓ サービスを起動し、確認を行った後は、必ず都度サービスを停止させる
- ✓ タスク完了後のアーキテクチャは以下の通りです



タスクの流れ

以下の流れをCloudFormationとTerraformでそれぞれ実施ください。

- 6-1. ネットワークレイヤーを記述する
- 6-2. アプリケーションレイヤーを記述する
- 6-3. データベースレイヤーを記述する

タスクの要件

共通

- CloudFormationを利用する場合はリソース名のxxxを「cfn」に置き換える
- Terraformを利用する場合はリソース名のxxxを「tf」に置き換える

6-1. ネットワークレイヤーを記述する

VPC

- リージョンは東京 (ap-northeast-1) を利用する
- VPCの名称は「xxx-vpc」とする
- VPCのCIDRブロックは 10.0.0.0/16とする
- DNS ホスト名を有効化する
- DNS 解決を有効化する

サブネット

- 以下の4つのサブネットを作成する
 - xxx-public-subnet1
 - ✧ ap-northeast-1a
 - ✧ 10.0.1.0/24
 - xxx-public-subnet2
 - ✧ ap-northeast-1c
 - ✧ 10.0.2.0/24
 - xxx-private-subnet1
 - ✧ ap-northeast-1a
 - ✧ 10.0.3.0/24
 - xxx-private-subnet2
 - ✧ ap-northeast-1c
 - ✧ 10.0.4.0/24

ルートテーブル

- 「xxx-public-rtb」という名前でパブリックルートテーブルを作成しパブリックサブネットに関連付ける
- 「xxx-private-rtb」という名前でプライベートルートテーブルを作成しプライベートサブネットに関連付ける

インターネットゲートウェイ

- 「xxx-igw」という名前でインターネットゲートウェイを作成しVPCに関連付ける
- デフォルトルート (0.0.0.0/0) をパブリックルートテーブルに紐づける

セキュリティグループ

- 「ec2-xxx-sg」という名前でEC2用のセキュリティグループを作成する
 - インバウンドルールとして HTTP (80番ポート) を全て許可する
 - インバウンドルールとしてSSH (22番ポート) の「3.112.23.0/29」からのアクセスを許可する
- 「rds-xxx-sg」という名前でRDS用のセキュリティグループを作成する
 - インバウンドルールとして MySQL (3306番ポート) をEC2セキュリティグループからのみ許可

出力

- VPC IDを出力する
- 各サブネットのサブネットIDを出力する
- EC2用のセキュリティグループIDを出力する
- RDS用のセキュリティグループIDを出力する

6-2. アプリケーションレイヤーを記述する

パラメーター (CloudFormationの場合のみ)

- ネットワークレイヤーのテンプレートを実行した際のスタック名「Network-Stack」を指定する

IAMロール

- EC2インスタンスが利用できるように設定する
- 「EC2 Instance Connect」ポリシーをアタッチする
- ロール名は「ec2-xxx-role」とする

EC2インスタンス

- IAMロールは「ec2-xxx-role」を利用する
- AMIは「Amazon Linux 2023」を利用する
- インスタンスタイプは「t3.micro」を利用する
- ネットワーク構成：
 - パブリックIP割り当てを有効化する
 - タスク6-1で作成したEC2用のセキュリティグループを使用する
 - タスク6-1で作成したパブリックサブネットを利用する
- インスタンス名は「ec2-xxx-instance」とする
- ユーザーデータ：

```
#!/bin/bash
# システムパッケージをアップデート
sudo dnf update -y
# nginx と必要なパッケージをインストール
sudo dnf install -y nginx mysql ncurses-compat-libs
# nginx を起動し、自動起動を有効化
sudo systemctl enable --now nginx
```

出力

- EC2インスタンスのIDを出力する

6-3. データベースレイヤーを記述する

パラメーター（CloudFormationの場合のみ）

- ネットワークレイヤーのテンプレートを実行した際のスタック名「Network-Stack」を指定する

DBサブネットグループ

- 「xxx-db-subnetgroup」という名前と説明にする
- タスク6-1で作成した2つのプライベートサブネットを利用する

RDS

- 「xxx-rds」という名前にする
- DBエンジンは「MySQL」の「8.0.41」を利用する
- インスタンスタイプは「db.t3.micro」を利用する
- ストレージタイプは「gp2」の「20GB」とする
- ネットワーク設定
 - DBサブネットグループは「xxx-db-subnetgroup」を利用する
 - タスク6-1で作成したDB用セキュリティグループを利用する
- 可用性/保護設定
 - デプロイオプションはシングルAZを利用する
 - マスターユーザー名は「admin」、パスワードは「password」とする
 - 自動バックアップはオフとする
 - マルチAZは無効とする
 - パブリックアクセスは無効とする
 - 削除保護は無効とする
 - 自動バックアップ削除は無効とする
 - 自動マイナーバージョンアップグレードは無効とする

- バックアップ保持期間は「0」（日）とする
- スナップショットへのタグコピーは無効とする

出力

- RDSインスタンスのIDを出力する
- RDSのエンドポイントを出力する

動作検証

- 作成したコードを実行してリソースが作成されることを確認する
 - CloudFormationとTerraformでそれぞれ実行する

後片付け

以上で演習は終了となります。

演習内で作成したリソースは停止や削除を行い、コストを節約することを徹底してください。

「チェックシート」での確認も併せてご実施ください。